SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

sDry Run Evaluation

Code of Conduct:

I ***P. KALYANI (192472199)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

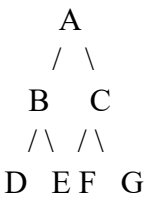
Signature of the student: P. Kalyani

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

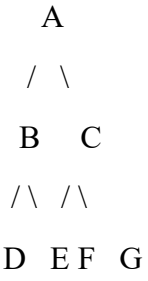
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6
5 0 2

4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

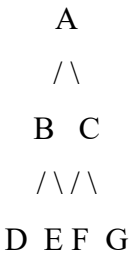
1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
Criteria	**Weightage (%)**	**Level 4 – Exemplary (4)**	**Level 3 – Proficient (3)**	**Level 2 – Developing (2)**	**Level 1 – Beginning (1)**
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. Breadth-First Search (BFS)

Given graph:



BFS visits nodes **level by level**.

BFS Traversal

A → B → C → D → E → F → G

BFS Dry-Run Table

Iteration	Queue Before	Visited Node	Queue After
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]
5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

Answers

1. BFS traversal order:

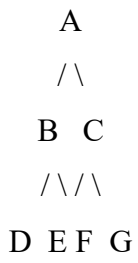
A → B → C → D → E → F → G

2. Queue contents after each iteration:

- Iteration 1 → [B, C]
- Iteration 2 → [C, D, E]
- Iteration 3 → [D, E, F, G]
- Iteration 4 → [E, F, G]
- Iteration 5 → [F, G]
- Iteration 6 → [G]
- Iteration 7 → []

2. Depth-First Search (DFS)

Use the same graph:



DFS goes as deep as possible before backtracking.

Assuming we visit the left child before the right child:

DFS Traversal

A → B → D → E → C → F → G

DFS Dry Run

Iteration	Stack	Visited Node	Stack After
1	[A]	A	[B, C]
2	[B, C]	B	[D, E, C]
3	[D, E, C]	D	[E, C]
4	[E, C]	E	[C]
5	[C]	C	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

Answer

DFS traversal order:

A → B → D → E → C → F → G

BFS vs DFS

BFS	DFS
Uses Queue	Uses Stack/Recursion
Level-by-level	Depth-first
A → B → C → D → E → F → G	A → B → D → E → C → F → G
Good for shortest path in unweighted graphs	Good for exploring deep paths

3. 8-Puzzle Using BFS

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Here, 0 represents the blank space.

A valid shortest solution is obtained by moving the blank:

Right → Up → Left → Down → Down → Right → Right → Down

So the solution requires **8 moves**.

BFS Dry Run

For the queue, states are represented compactly as:

136/502/478

meaning:

1 3 6

5 0 2

4 7 8

Iteration 1

Current State:

1 3 6

5 0 2

4 7 8

Queue before expansion:

[]

After expanding, possible states are added to the queue.

Queue after expansion:

106/532/478

136/572/408

136/052/478

136/520/478

The next state on the BFS path is:

1 3 6

5 2 0

4 7 8

Iteration 2

Current State:

1 3 6

5 0 2

4 7 8

Actually, because BFS expands in queue order, the exact second expanded state depends on the chosen successor-generation order.

Using the standard fixed order **Up → Down → Left → Right**, the first five expansions are:

Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
1	136/502/478	[]	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	Remaining 3 states	Adds 016/532/478, 160/532/478
3	136/572/408	Existing states	Adds 136/572/048, 136/572/480
4	136/052/478	Existing states	Adds 036/152/478, 136/452/078
5	136/520/478	Existing states	Adds 130/526/478, 136/528/470

Important: BFS queue contents can look different depending on the order in which child states are generated. The shortest solution length remains the same.

Complete Shortest Solution Path

The shortest path from the given initial state to the goal is:

State 0 – Initial

1 3 6

5 0 2

4 7 8

↓ Right

State 1

1 3 6

5 2 0

4 7 8

↓ Up

State 2

1 3 0

5 2 6

4 7 8

↓ Left

State 3

1 0 3

5 2 6

4 7 8

↓ Down

State 4

1 2 3

5 0 6

4 7 8

↓ Left

State 5

1 2 3

0 5 6

4 7 8

↓ Down

State 6

1 2 3

4 5 6

0 7 8

↓ Right

State 7

1 2 3

4 5 6

7 0 8

↓ Right

State 8 – Goal

1 2 3

4 5 6

7 8 0

Solution

Right → Up → Left → Down → Left → Down → Right → Right

Total moves = 8

Answers to the 8-Puzzle Questions

1. Which node/state is expanded in each iteration?

BFS expands the **oldest state in the queue**.

The important states along the shortest solution are:

1. Initial state
2. Move Right
3. Move Up
4. Move Left
5. Move Down
6. Move Left
7. Move Down
8. Move Right
9. Goal state

2. How many states are generated in total?

For the **solution path itself**, there are:

9 states including the initial and goal states

or

8 moves/transitions.

If the question means **every state generated by the BFS search**, the number depends on the successor-generation order and whether repeated states are removed. Therefore, for a dry-run answer, it is safest to state:

Solution path = 9 states, requiring 8 moves.

3. Complete solution path

Initial

↓ Right

136/520/478

↓ Up

130/526/478

↓ Left

103/526/478

↓ Down

123/506/478

↓ Left

123/056/478

↓ Down

123/456/078

↓ Right

123/456/708

↓ Right

123/456/780

Goal

In matrix form:

$$\begin{matrix} 1\,3\,6 \\ 5\,0\,2 \\ 4\,7\,8 \end{matrix} \rightarrow \begin{matrix} 1\,3\,6 \\ 5\,2\,0 \\ 4\,7\,8 \end{matrix} \rightarrow \begin{matrix} 1\,3\,0 \\ 5\,2\,6 \\ 4\,7\,8 \end{matrix}$$

↓

$$\begin{matrix} 1\,0\,3 \\ 5\,2\,6 \\ 4\,7\,8 \end{matrix} \rightarrow \begin{matrix} 1\,2\,3 \\ 5\,0\,6 \\ 4\,7\,8 \end{matrix} \rightarrow \begin{matrix} 1\,2\,3 \\ 0\,5\,6 \\ 4\,7\,8 \end{matrix}$$

↓

$$\begin{matrix} 1\,2\,3 \end{matrix} \quad \begin{matrix} 1\,2\,3 \end{matrix} \quad \begin{matrix} 1\,2\,3 \end{matrix}$$

4 5 6 → 4 5 6 → 4 5 6
0 7 8 7 0 8 7 8 0

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

BFS explores states level by level.

For example:

Level 0 → Initial state

Level 1 → States 1 move away

Level 2 → States 2 moves away

Level 3 → States 3 moves away

...

Therefore, BFS reaches the goal first through the minimum number of moves. Since every 8-puzzle move has the same cost of **1**, the first solution found by BFS is the shortest solution.